

UNIVERSIDAD CENTRAL DE VENEZUELA  
FACULTAD DE CIENCIAS  
ESCUELA DE COMPUTACIÓN  
DISEÑO Y PROTOTIPADO DE RECURSOS EDUCATIVOS DIGITALES (DPRED)

---

# PROPUESTA DE RECURSO EDUCATIVO DIGITAL: APRENDE GIT INTERACTIVO

*"Plataforma de Simulación Visual y Tutoría Inteligente para la Enseñanza y Comprensión del  
Control de Versiones"*

INFORME DE INVESTIGACIÓN, ANÁLISIS Y DISEÑO (ENTREGA #1)

Autores - Grupo #4:

Jonathan Torres

Ricardo Riera

Javier Darder

Docentes de Cátedra:

Prof. Felix Urbano

Profa. Yusneyi Carballo Barrera

Caracas, 25 de junio de 2026

# 1. Definición del Problema y su Alcance

En el ámbito de la ingeniería de sistemas, ciencias de la computación y desarrollo de software, el control de versiones utilizando Git se ha consolidado como una competencia profesional básica no negociable. Sin embargo, existe una desconexión crítica entre el aprendizaje académico de la herramienta y las habilidades requeridas en entornos laborales reales.

Se estima que el 80% de los estudiantes de carreras informáticas llegan a su primer empleo o a las asignaturas de desarrollo colaborativo (como Ingeniería de Software) sin comprender el modelo mental subyacente de Git. El aprendizaje tradicional se limita a la memorización mecánica de una secuencia de comandos sintácticos (como `git add`, `git commit`, `git push`) sin que el estudiante comprenda el flujo de estados internos, la estructura del Grafo Acíclico Dirigido (DAG) o las implicaciones arquitectónicas de sus acciones. Esta falta de entendimiento genera lo que en la industria se conoce como 'ansiedad ante la terminal' (CLI anxiety) y deriva en errores críticos en proyectos grupales, como la pérdida accidental de código o la parálisis del flujo de trabajo ante el primer conflicto de integración.

## 1.1. Delimitación y Alcance del Recurso Educativo Digital (RED)

Para transformar este paradigma instructivo, el RED 'Aprende Git Interactivo' se plantea como una plataforma educativa web estructurada en tres grandes módulos interactivos, diseñados bajo el enfoque de gamificación y Diseño Universal para el Aprendizaje (DUA):

- **Módulo 1: El Core - Visualización del Entorno Local y Grafo Vivo.** Visualiza de manera dinámica la separación conceptual y la transferencia de datos entre el Working Directory (directorio de trabajo), el Staging Area (zona de preparación) y el Local Repository (repositorio local). Cada comando ejecutado en la consola simulada dibuja y manipula en tiempo real un Grafo Vivo de commits y referencias (ramas y puntero HEAD), permitiendo que el usuario 'vea' la estructura interna de Git.
- **Módulo 2: El Puente - Conexión con GitHub y Mecánicas Colaborativas.** Introduce la conexión con servidores remotos. Simula los flujos de sincronización (push, fetch y pull) representando al servidor como un segundo grafo espejo. Asimismo, gamifica dinámicamente las mecánicas de creación de ramas de características (feature branches), simulación de Pull Requests (PR) y procesos de Code Review con respuestas predefinidas.
- **Módulo 3: El Modo Supervivencia - Resolución de Conflictos e Historiales Rotos.** Aborda directamente la resolución de conflictos. Contiene un simulador interactivo de resolución de conflictos con una interfaz simplificada que imita los marcadores visuales de conflicto de editores modernos (como VS Code). Además, simula y contrasta en paralelo los conceptos de Merge vs. Rebase para que el alumno visualice cómo el rebase reescribe linealmente la historia y cuándo resulta peligroso.
- **Tutor Pedagógico con Inteligencia Artificial (Nova):** Se implementará un tutor pedagógico basado en inteligencia artificial (Nova) integrado directamente en el frontend. Si el estudiante permanece estancado por más de 2 minutos en un nivel o comete errores consecutivos, Nova se activará

discretamente para explicar el origen lógico del conflicto o desalineación del árbol, ofreciendo andamiaje cognitivo en lugar de resolver la tarea por el usuario.

Exclusiones del Proyecto: El RED no busca emular la totalidad de las opciones avanzadas de Git (como comandos complejos de git reflog, filter-branch o hooks de servidor), sino asegurar el dominio sólido de los flujos de trabajo profesionales comunes (Gitflow y Trunk-based Development) para estudiantes y desarrolladores junior.

## 1.2. Matriz FODA (Fortalezas, Oportunidades, Debilidades, Amenazas)

A continuación se presenta la matriz FODA del proyecto para evaluar factores internos controlables y externos del entorno:

| ORIGEN INTERNO (Controlable)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                | ORIGEN EXTERNO (Entorno)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>FORTALEZAS (F)</b> <ul style="list-style-type: none"><li>1. Interactividad Real: Enfoque práctico y visual que supera los tutoriales pasivos de video.</li><li>2. Arquitectura Client-Side: Simulación del motor Git en el navegador del cliente (JS/WebAssembly), reduciendo costos de hosting a cero.</li><li>3. Tutoría Inteligente Integrada: Asistencia contextual basada en andamiaje con IA (Nova) en atascos.</li><li>4. Enfoque DUA: Accesibilidad planificada desde el inicio para inclusión educativa.</li></ul>                                              | <b>OPORTUNIDADES (O)</b> <ul style="list-style-type: none"><li>1. Alta Demanda Sector Tech: Necesidad obligatoria de aprender control de versiones en el currículo de sistemas.</li><li>2. Adopción Académica UCV: Oportunidad de ser utilizado como herramienta de apoyo formal en Ingeniería de Software.</li><li>3. Comunidad de Software Libre: Atracción de desarrolladores colaboradores al liberar el código en GitHub.</li><li>4. Alianzas y Créditos: Obtención de facilidades gratuitas de despliegue mediante programas como GitHub Education.</li></ul>                                                           |
| <b>DEBILIDADES (D)</b> <ul style="list-style-type: none"><li>1. Presupuesto Limitado: Al ser un proyecto de cátedra, se carece de fondos comerciales para despliegue avanzado.</li><li>2. Restricción de Tiempo Académico: Cronograma acotado de un semestre para implementar la complejidad del motor de Git.</li><li>3. Dependencia del Equipo: Avance limitado a las horas y habilidades disponibles del grupo de desarrollo.</li><li>4. Curva de Desarrollo UI/UX: Complejidad técnica para hacer una emulación de consola que sea 100% amigable y accesible.</li></ul> | <b>AMENAZAS (A)</b> <ul style="list-style-type: none"><li>1. Competidores Consolidados: Sitios con alto posicionamiento internacional (ej. Learnitbranching, Oh My Git!).</li><li>2. Políticas de API de GitHub: Modificaciones externas drásticas en OAuth de GitHub que invaliden el flujo del Módulo 2.</li><li>3. Frustración Temprana del Alumno: Abandono de la plataforma si los niveles avanzados o la consola virtual resultan muy difíciles.</li><li>4. Herramientas Automatizadas de IA: Evolución de asistentes de IA que resuelven conflictos solos, reduciendo el interés del estudiante en aprender.</li></ul> |

## 2. Análisis del Entorno (PESTEL)

Para asegurar la viabilidad de la plataforma como un proyecto académico e innovador y evaluar el impacto del entorno, se desarrolla el análisis PESTEL correspondiente:

- **Dimensión Política (P):** La promoción nacional y regional de la educación STEM y la alfabetización digital. La plataforma encaja como un recurso de apoyo didáctico para universidades técnicas. Existe un riesgo geopolítico bajo de restricciones en los servicios de infraestructura cloud usuales (Vercel, Firebase).
- **Dimensión Económica (E):** El elevado costo de la educación técnica formal potencia la demanda de herramientas educativas libres de alta calidad. La principal restricción económica radica en los costos de infraestructura (servidores interactivos en USD). Para solucionar esto, la arquitectura se diseñará para ejecutar la simulación de Git directamente en el navegador del cliente, reduciendo los costos de computación central a cero.
- **Dimensión Social (S):** Las nuevas generaciones exigen formatos interactivos y gamificados ('Play-and-Learn'), rechazando manuales estáticos en PDF o videos instructivos largos y pasivos. La interfaz debe diseñarse con alta empatía cognitiva para reducir la frustración inicial de interactuar con una consola de comandos.
- **Dimensión Tecnológica (T):** La madurez de JavaScript y WebAssembly permite emular un sandbox del sistema de archivos y el motor de Git 100% en el frontend del cliente. Se requiere la integración mediante la API de GitHub (OAuth) para posibilitar que el estudiante conecte opcionalmente su portafolio real.
- **Dimensión Ecológica / Ambiental (E):** La optimización de código del lado del cliente (frontend-first) contribuye al Green Computing al disminuir el uso innecesario de ciclos de CPU y consumo energético en centros de datos. Elimina al 100% la huella ecológica asociada a guías impresas, evaluaciones físicas o manuales de laboratorio.
- **Dimensión Legal (L):** Obligatoriedad de cumplir con normativas de protección de datos (especialmente al almacenar sesiones u OAuth con tokens de GitHub). Es legalmente indispensable incorporar un descargo de responsabilidad (disclaimer) que aclare que el recurso es independiente y no está patrocinado de manera oficial por Microsoft o GitHub.

### 3. Análisis de Competidores (Benchmarking)

El ecosistema de recursos para aprender Git se divide principalmente en herramientas operativas o tutoriales teóricos pasivos. Para posicionar estratégicamente a 'Aprende Git Interactivo', se realizó un benchmarking competitivo:

**Competidor Directo: Learngitbranching.js.org.** Es el referente directo más exitoso en la web. Ofrece un entorno excelente con una terminal de comandos interactiva y un árbol visual en tiempo real. No obstante, presenta deficiencias notables: carece de simulación de flujos de trabajo colaborativos reales (como la revisión de código en Pull Requests con interfaz similar a la de GitHub), no cuenta con un editor interactivo para resolver conflictos línea por línea, carece de tutoría con Inteligencia Artificial que explique conceptualmente los atascos y su diseño web no es accesible para estudiantes con discapacidad visual o de aprendizaje (incumplimiento del Diseño Universal para el Aprendizaje - DUA).

**Competidor Directo Secundario: Oh My Git!** Juego de escritorio open-source para aprender Git. Es muy visual y cuenta con mecánicas de juego atractivas, pero al requerir instalación física no es apto para un acceso inmediato en salas de computación universitaria o teléfonos móviles. No simula la interfaz web de GitHub.

**Competidores Indirectos (Herramientas Visuales):** Clientes visuales como GitKraken, Sourcetree o GitHub Desktop. Aunque ayudan al programador junior a evitar la consola, son herramientas de producción y no plataformas educativas. El usuario aprende a presionar botones pero no a entender conceptualmente el flujo, perpetuando su dependencia tecnológica.

**Competidores Indirectos (Contenido Pasivo):** Cursos en video de plataformas de e-learning (Udemy, YouTube) y la documentación oficial de git-scm. Son recursos fundamentalmente pasivos de alto nivel teórico y nula interactividad práctica, propensos a generar aburrimiento y deserción temprana.

#### Matriz de Benchmarking Competitivo

| Competidor                          | Simulación de Grafo         | Mecánica Remota / PR      | Editor de Conflictos        | Tutoría Inteligente (IA) | Accesibilidad (DUA) | Disponibilidad           |
|-------------------------------------|-----------------------------|---------------------------|-----------------------------|--------------------------|---------------------|--------------------------|
| Aprende Git Interactivo (Propuesta) | Sí, animado interactivo     | Sí, simulación web de PRs | Sí, editor línea por línea  | Sí, Nova AI integrada    | Sí, WCAG AA / DUA   | Web/Móvil (Cero costo)   |
| Learngitbranching.js.org            | Sí, interactivo de comandos | No, comandos mock simples | No, automático por comandos | No                       | No accesible        | Web (Cero costo)         |
| Oh My Git!                          | Sí, vista gráfica en juego  | No, simulación local      | No, automático por UI       | No                       | Medio               | Escritorio (Instalación) |

|                                     |                             |                               |                                 |                               |      |                      |
|-------------------------------------|-----------------------------|-------------------------------|---------------------------------|-------------------------------|------|----------------------|
| Clientes<br>Gráficos<br>(GitKraken) | Sí, lectura de<br>repo real | Sólo links e<br>integraciones | Sí,<br>herramientas<br>de merge | No educativa<br>(Herramienta) | Bajo | Escritorio /<br>Pago |
|-------------------------------------|-----------------------------|-------------------------------|---------------------------------|-------------------------------|------|----------------------|

## 4. Análisis del Cliente (User Persona)

Para diseñar una solución centrada en el usuario, se modeló el perfil psicográfico del cliente objetivo con base en las realidades observadas en los primeros niveles de las carreras tecnológicas de la Universidad Central de Venezuela:

**Perfil del Arquetipo:** Carlos Mendoza (El Estudiante de Tecnología Abrumado)

- **Edad:** 21 años.
- **Ocupación:** Estudiante de los primeros semestres de Licenciatura en Computación en la UCV y desarrollador junior autodidacta.
- **Contexto Educativo:** Posee bases de programación conceptual. Sin embargo, carece de experiencia en el trabajo colaborativo en equipos y el uso formal del control de versiones. La terminal de sistema operativo le genera una barrera psicológica importante.

**Trabajos del Cliente (Customer Jobs):** Aprobar la asignatura de desarrollo colaborativo de software, coordinar de manera fluida y sin tropiezos la integración del código de su grupo de proyecto, y desarrollar el portafolio personal en GitHub para aspirar a sus primeras pasantías o empleos.

**Frustraciones (Pains):** La complejidad sintáctica y falta de feedback visual de la terminal, el miedo paralizante de cometer un error (como git push --force o una fusión fallida) y arruinar el trabajo acumulado del equipo, la naturaleza densa y teórica de los manuales de Git, y la falta de traducción en español de los errores más comunes.

**Alegrías (Gains):** Disponer de un sandbox interactivo y seguro donde poder equivocarse y experimentar sin miedo a perder archivos reales, recibir retroalimentación correctiva al instante, entender con diagramas de nodos dinámicos qué representa cada comando ejecutado y sentir un progreso motivador a través de mecánicas de juego estructuradas.

### 4.2. User Journey Map (Mapa de Viaje del Usuario)

A continuación se modela el flujo de experiencia de Carlos Mendoza al interactuar con el recurso interactivo:

| Etapas            | Acciones del Usuario | Puntos de Contacto | Pensamientos/ Emociones | Puntos de Dolor     | Oportunidades en RED |
|-------------------|----------------------|--------------------|-------------------------|---------------------|----------------------|
| 1. Descubrimiento | Entra a la           | Enlace en          | Curioso,                | Enlace caído, carga | Garantizar carga     |

|                            |                                                                               |                                                          |                                                                           |                                                               |                                                                            |
|----------------------------|-------------------------------------------------------------------------------|----------------------------------------------------------|---------------------------------------------------------------------------|---------------------------------------------------------------|----------------------------------------------------------------------------|
|                            | plataforma por recomendación de su profesor en el Aula Virtual UCV.           | Classroom / Web pública.                                 | expectante. Siente flojera por tener que leer teoría.                     | inicial de la web lenta.                                      | ultra-rápida (menos de 2s) y diseño visual atractivo.                      |
| <b>2. Onboarding</b>       | Inicia sesión con su cuenta GitHub y realiza un test rápido diagnóstico.      | OAuth de GitHub. Interfaz de bienvenida.                 | Nervioso por conectar su cuenta personal. Intrigado por el nivel inicial. | Exceso de permisos requeridos por OAuth.                      | Solicitar permisos mínimos de lectura pública para su portafolio.          |
| <b>3. Aprendizaje Core</b> | Completa comandos básicos de Git local y observa los nodos moverse.           | Consola y panel del Grafo Vivo.                          | Motivado y con confianza. '¡Ah, ya entiendo qué hace git commit!'         | Errores tipográficos en la consola simulada.                  | Añadir autocompletado y sugerencias sintácticas rápidas.                   |
| <b>4. Conflicto</b>        | En el módulo 3, el sistema le provoca un conflicto al fusionar ramas.         | Interfaz del simulador con archivo de conflicto en rojo. | Frustrado, ansioso. Siente miedo a romper las ramas del equipo.           | Mensaje tosco de conflicto en pantalla, sensación de bloqueo. | Habilitar botón de reset de nivel inmediato para disminuir la frustración. |
| <b>5. Andamiaje IA</b>     | Permanece 2 minutos bloqueado. El bot Nova IA le abre una ventana con pistas. | Chat flotante y nodos resaltados en el grafo.            | Aliviado, enfocado. 'Por fin alguien me explica por qué chocó esto'.      | Explicación muy larga o genérica que no ayuda.                | Hacer respuestas de Nova ultra-cortas y apoyadas en el grafo visual.       |
| <b>6. Logro y Cierre</b>   | Resuelve la colisión de código, avanza de nivel y comparte su certificado.    | Certificado digital en la web. Botón de LinkedIn.        | Orgulloso, con mayor autoeficacia técnica.                                | Falta de incentivo real para exportar o mostrar su progreso.  | Permitir exportar los ejercicios resueltos a su GitHub real como commits.  |

## 5. Propuesta de Solución y Justificación

La solución propuesta es el Recurso Educativo Digital 'Aprende Git Interactivo' (GitMaster). Se plantea como una plataforma Web interactiva de simulación y andamiaje cognitivo que aborda de raíz las frustraciones detectadas y satisface los trabajos de aprendizaje de Carlos Mendoza de manera eficiente y ágil.

### 5.1. Justificación Pedagógica y de Diseño Educativo

- **Teoría Constructivista y Modelado Mental:** La plataforma se cimenta sobre la teoría del constructivismo interactivo (Novak & Gowin). En lugar de la memorización verbal del comando, el

estudiante manipula directamente el Grafo Vivo. Al cambiar las ramas, visualiza cómo se mueven los punteros locales de Git. Así, el alumno autoconstruye el modelo mental de Git como un sistema de almacenamiento de instantáneas (commits) y referencias dinámicas, lo que asegura un aprendizaje significativo a largo plazo.

- **Inclusión y Accesibilidad Universal:** Alineado con el Diseño Universal para el Aprendizaje (DUA) e incorporando los estándares de accesibilidad WCAG AA. Esto implica el uso de contrastes de color validados (evitando depender solo del color para diferenciar ramas), descripciones de voz alternativas y soporte de navegación por teclado completa para que el simulador sea inclusivo en el aula.
- **Sostenibilidad y Gamificación:** La simulación local en el frontend (reducción de costos energéticos y de hosting), y la gamificación como potenciador de la constancia estudiantil para mitigar la deserción temprana.

## 5.2. Mockup Conceptual de la Interfaz del Prototipo

A continuación se presenta el prototipo visual de la interfaz del RED, diseñado para optimizar la carga cognitiva del usuario y mantener el andamiaje del tutor inteligente de IA (Nova) visible e interactivo:

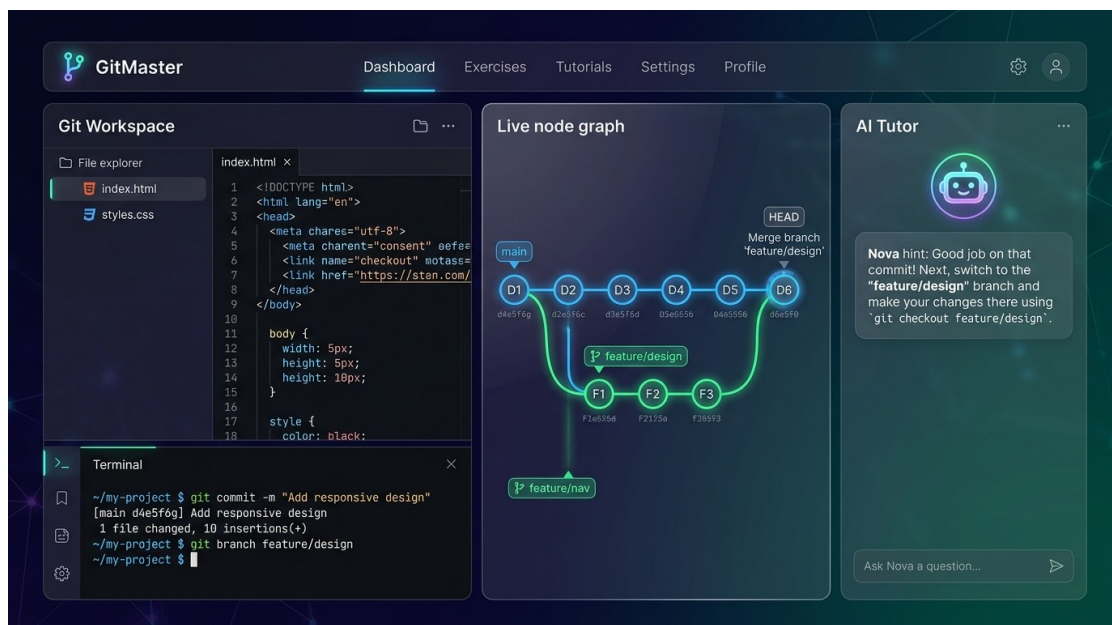


Figura 1: Mockup conceptual de la plataforma 'Aprende Git Interactivo' en modo oscuro. Se visualiza a la izquierda el espacio de código y terminal simulada; en el centro, el Grafo Vivo del repositorio; y a la derecha, la asistencia de andamiaje pedagógico de la tutora Nova IA.

## 6. Encaje de Valor (Value Proposition Canvas)

El mapa de encaje de valor analiza el acoplamiento directo entre los módulos del Recurso Educativo Digital (RED) y las necesidades psicográficas de nuestro usuario arquetípico Carlos Mendoza:

- **1. Productos y Servicios del RED:** Web App interactiva con terminal virtual emulada en JS, editor visual de archivos de código con resaltador de sintaxis, simulador dinámico del Grafo Vivo del repositorio en SVG/Canvas, y chat de tutoría contextual asistida por IA (Nova).
- **2. Aliviadores de Frustraciones (Pain Relievers):** Entorno cerrado de simulación (sandbox) que elimina el riesgo de borrar archivos reales del equipo, traducción de códigos de error de consola a explicaciones amigables en español, pistas contextuales automáticas basadas en IA tras periodos de inactividad, y niveles de andamiaje progresivos (init -> conflict solver).
- **3. Creadores de Alegrías (Gain Creators):** Actualización inmediata del árbol gráfico de commits y ramas con cada comando, sistema de progresión gamificado (puntuación, ligas académicas virtuales y medallas), simulaciones interactivas de fusión colaborativa (PRs) con roles automatizados de revisores, y explicaciones intuitivas sobre Merge vs. Rebase.

El encaje de valor es robusto y se manifiesta de forma directa en tres intersecciones clave del diseño educativo y técnico:

- **Intersección Sandbox - Reducción del Miedo:** El temor de Carlos a arruinar repositorios compartidos se neutraliza por completo con el simulador del lado del cliente. Al tratarse de un entorno sandbox local, el estudiante puede experimentar libremente, ejecutar rebase destructivos o borrar punteros HEAD sin consecuencias reales, construyendo resiliencia ante el error.
- **Intersección Grafo Vivo - Superación de CLI Anxiety:** El atasco cognitivo de Carlos frente a la consola se mitiga con la visualización gráfica inmediata. Si ejecuta 'git checkout -b feature', observa en milisegundos cómo nace una rama derivada del commit actual y cómo el puntero HEAD se traslada al nuevo nodo. Esto transforma el comando ciego en un movimiento espacial comprensible.
- **Intersección Nova IA - Andamiaje Pedagógico:** La tutoría contextual de Nova IA llena el vacío de los tutoriales pasivos. Cuando un conflicto ocurre, en lugar de recibir un mensaje genérico de rechazo no-fast-forward, la IA guía la atención de Carlos hacia los nodos divergentes en el gráfico, propiciando que deduzca por sí mismo por qué los commits del remoto no coinciden y cómo resolver el conflicto de integración.

## 7. Fuentes Consultadas y Referencias Bibliográficas

- Álvarez, L., Carballo Barrera, Y., Collazos, C., Echenagusía, J., Gutiérrez, R., Hernández Bieliukas, Y., Muñoz, J., Solano, A., y Velázquez, C. (2015). *Objetos de Aprendizaje de Contenidos Abiertos Accesibles: del Diseño a la Reutilización*. Iniciativa Latinoamericana de Libros de Texto Abiertos (LATIn). Licencia CC BY-SA 3.0.
- Caeiro-Rodríguez, M., Costa-Montenegro, E., Díaz-Otero, F., Cuiñas-Gómez, I., Mariño-Espiñeira, P., y Fernández-Iglesias, M. (2016). *Evaluación de la implantación de la metodología Design Thinking en una asignatura de proyectos*. Congreso In-Red 2016 UPV. Valencia, España.
- Copley, J. (s.f.). *Learn Git Branching (Simulador interactivo web de ramas y commits)*. Disponible en <https://learngitbranching.js.org>
- Del Prado, J. (2024). *Design Thinking, Waterfall, Agile y Lean UX. Cuál usar*. Uxables Blog.
- Díaz-Barriga, F. y Hernández, G. (2002). *Estrategias Docentes para un Aprendizaje Significativo: una interpretación constructivista*. McGraw-Hill. México.
- Git Project Authors. (s.f.). *Git - Reference Manual and Documentation*. Disponible en <https://git-scm.com/doc>
- Gros, Begoña. (1997). *Diseño y Programas educativos. Pautas pedagógicas para la elaboración de software*. Editorial Ariel. España.
- Herrera, M. (2006). *Consideraciones para el diseño didáctico de ambientes virtuales de aprendizaje: Una propuesta basada en las funciones cognitivas del aprendizaje*. Revista Iberoamericana de Educación, 38(5).
- IBM Cloud Education. (2022). *Low-Code vs. No-Code: What's the difference?* Recuperado de <https://www.ibm.com/think/topics/low-code-vs-no-code>
- Lodi, M. y Martini, S. (2021). *Computational Thinking, Between Papert and Wing*. Science & Education, 30(4), 883-908. DOI: 10.1007/s11191-021-00202-5.
- Novak, J. D. y Gowin, D. (1988). *Aprendiendo a aprender*. Editorial Martínez Roca. Barcelona, España.
- Ospina, M. y Carballo Barrera, Y. (2022). *Revisión de Técnicas y Tecnologías para la Construcción del Producto Mínimo Viable en el Contexto del Emprendimiento Universitario*. Novena Conferencia Nacional de Computación, Informática y Sistemas (CoNCISa 2022). Caracas, Venezuela.
- Schneiderman, B. y Plaisant, C. (2006). *Diseño de interfaces de usuario: Estrategias para la interacción persona-computadora*. 4ta Ed. Addison Wesley.
- W3C. (2021). *Resumen de los estándares de accesibilidad de W3C (Web Accessibility Initiative - WAI)*. Recuperado de <https://www.w3.org/WAI/standards-guidelines/es>
- Woolf, B. P. (2010). *Building Intelligent Interactive Tutors: Student-centered thinking in/about education*. Morgan Kaufmann. (Referente para sistemas de tutoría inteligente asistidos por IA).